

AI-Powered Hyper-Personalized Banking for Bharat

Technical Architecture & Approach Report

Theme: Digital Transformation in Lending

Hackathon Technical Submission — Draft for Team Review

Contents

1. Executive Summary
2. Problem Statement Recap
3. Design Philosophy & Guiding Principles
4. Unified System Architecture
5. Sub-Problem 1 — Hyper-Personalized Recommendation Engine
6. Sub-Problem 2 — Vernacular Conversational AI (Slot-Filling + RAG)
7. Sub-Problem 3 — Fraud & Financial Stress Detection
8. Cross-Cutting Decision Layer (Veto Logic)
9. Ethical Safeguards & Regulatory Compliance (DPDP / RBI)
10. Technology Stack Summary
11. USPs & Differentiators
12. Datasets
13. Build Roadmap & Prioritization
14. Judging Criteria Mapping
15. Known Limitations & Honest Gaps

1. Executive Summary

This system addresses the three sub-problems of the theme — proactive personalization, vernacular-first conversational access, and empathetic fraud/stress detection — as one connected decisioning platform rather than three independent features. The unifying idea is a shared behavioral feature pipeline feeding three specialist model heads (segmentation/propensity, anomaly detection, stress-trend detection), all gated through a single explainable veto layer before anything reaches the customer. This ensures that personalization, conversational assistance, and risk detection never contradict each other — a stressed or flagged customer cannot be pushed a loan offer through one channel while being flagged as at-risk through another.

The core differentiation is not model sophistication but decision discipline: every recommendation is explainable (SHAP-based), every credit-product push is subject to a hard affordability/stress floor (not just a soft discount), and every vernacular conversation is grounded in cited government/regulatory sources rather than free-form generation. This directly targets the judging emphasis on genuine customer benefit over upsell, and regulatory compliance readiness over raw technical novelty.

2. Problem Statement Recap

The challenge asks for an AI-powered solution across three connected capabilities:

- **Sub-problem 1:** Proactively recommend the right banking product (loan, insurance, investment, credit card) using transaction history, spending patterns, and life-stage signals — replacing generic pop-up offers.
- **Sub-problem 2:** Simplify onboarding, loan application, and KYC through conversational AI in vernacular languages, reducing drop-off for first-time digital users.
- **Sub-problem 3:** Detect early signals of financial stress or fraud and trigger proactive, empathetic interventions instead of purely punitive actions.

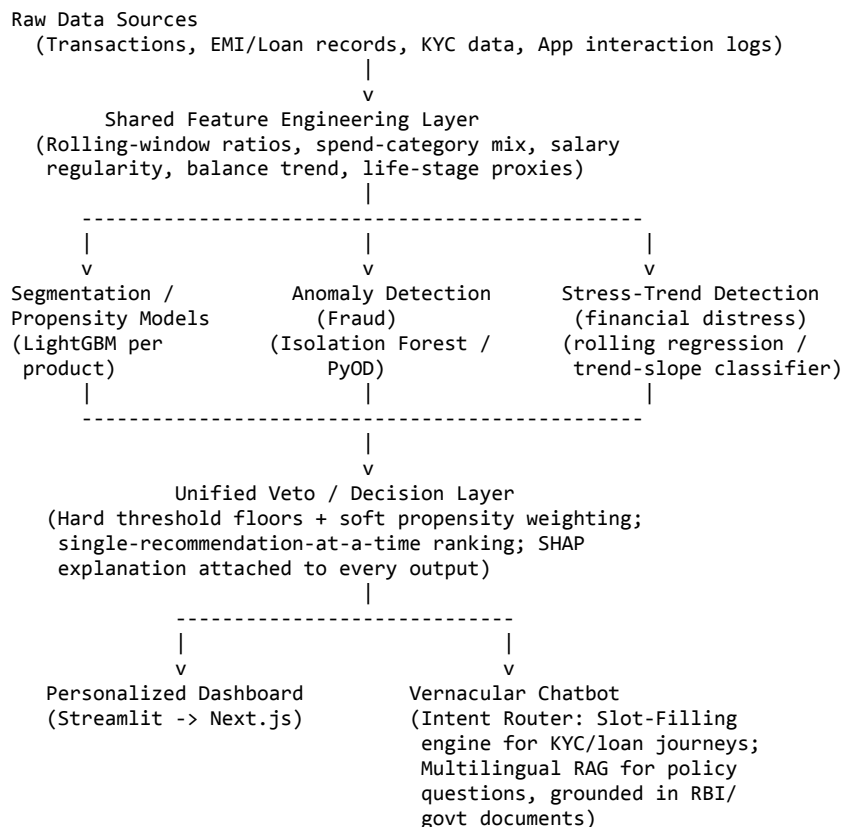
Judging weighs innovation, depth of genuine personalization (not just upsell), explainability/regulatory readiness, usability for non-tech-savvy vernacular-first users, and scalability across existing bank infrastructure.

3. Design Philosophy & Guiding Principles

- **One feature pipeline, multiple specialist models — not one monolith.** Personalization, fraud, and stress detection read different signal types (static/segment, point-anomaly, trend) from the same underlying transaction data. Forcing a single model to serve all three would compress three distinct optimization objectives into one loss function and weaken all three.
- **Explainability is not a bolt-on.** Every recommendation carries a SHAP-based, plain-language justification by construction, not as a post-hoc report generated for judges.
- **Hard floors, not just soft adjustments.** Wherever a decision could cause real financial harm (pushing credit to a stressed user), a deterministic rule-based veto exists beneath any ML-driven soft weighting. ML tunes the experience; it does not have unlimited authority over risk-sensitive actions.
- **Purpose-bound data, not maximal profiling.** Customer context is tiered by consent purpose (implicit account data, opt-in behavioral data, explicit opt-in life-stage data), aligned to DPDP Act's purpose-limitation principle, rather than one unrestricted profile object.
- **Single recommendation surfaced at a time.** The system deliberately avoids stacking multiple product offers, directly countering the "generic pop-up" pattern the problem statement flags as the status quo failure.

4. Unified System Architecture

The three sub-problems share infrastructure. The diagram below shows how data flows from raw sources through the shared feature layer, into three specialist model heads, through the unified veto/decision layer, and out through two customer-facing surfaces (dashboard and chatbot).



Scope note: Diagram above is the reference architecture for the pitch deck. Render it as a polished graphic (e.g., in Canva or draw.io) for the actual submission — this is the structural source of truth.

5. Sub-Problem 1 — Hyper-Personalized Recommendation Engine

5.1 Approach

Rejected pure collaborative filtering due to cold-start failure on first-time digital users with sparse interaction history. Adopted a two-layer approach: behavioral segmentation followed by per-product propensity ranking, gated by the affordability/stress veto layer (Section 8).

5.2 Data Tiers (Consent-Scoped Profile)

Tier	Data	Consent Basis	Used For
Tier 0 (Implicit)	Salary credit regularity, account vintage, KYC-declared occupation	Account relationship	All products
Tier 1 (Behavioral)	EMI history, spend-category mix, savings velocity, balance trend	Opt-in at onboarding	Segmentation, propensity scoring
Tier 2 (Explicit)	Life-stage signals (marriage, home purchase intent, dependents)	Per-purpose opt-in	Investment, insurance only

This tiering is deliberate: a maximal single "user profile" object would violate DPDP's purpose-limitation principle and increase proxy-discrimination risk (Section 9). Each recommendation is traceable to which tier justified it.

5.3 Feature Set (Shared Pipeline)

- Rolling EMI-to-income ratio (30/90/180-day windows)
- Savings rate and savings-rate trend
- Spend-category mix and category-share shift
- Salary-credit regularity and salary-jump detection
- New recurring debit detection (possible new EMI/rent — life-event proxy)
- Account vintage and balance-floor trend

5.4 Model Layer (Per Product, Not One Model)

Product	Model	Key Signal	Rationale
Personal Loan	LightGBM classifier (propensity)	EMI headroom, salary stability	Tabular data, needs SHAP explainability, fast to fit within hackathon time
Insurance	LightGBM classifier (separate target)	Life-stage signal, dependents, absence of existing cover	Distinct label from loan; feature weighting should not be reused across products
Investment	Rule-assisted scoring (not ML-first)	Savings surplus after EMI + essentials, risk-appetite proxy	Investment mis-selling is heavily regulated; transparent rules are easier to defend as non-predatory than a black-box model given no real risk-profile data
Credit Card	LightGBM classifier	Spend volume, repayment discipline, absence of existing card	Reuses most of loan's feature set with a different target label

5.5 Ranking & Output

Per-product propensity scores are adjusted by the stress-based soft weight (or hard-vetoed where the severity threshold is crossed — see Section 8), then ranked. Only the top 1–2 ranked products are surfaced at any time, each with a SHAP-derived plain-language explanation (e.g., "recommended because your savings grew 18% over 3 months and you have no existing insurance cover").

5.6 Build Priority

Loan and insurance models first (highest feature overlap with the stress/fraud veto layer and clearest explainability story). Investment (rules-based) and credit card are extensions to build only if time remains; both should still appear in the architecture diagram as "designed for" even if not fully implemented.

6. Sub-Problem 2 — Vernacular Conversational AI

6.1 Approach

Rejected the naive translate-input -> retrieve-in-English -> translate-output pipeline. This architecture compounds translation error at three separate points and is especially unreliable on financial terminology that lacks clean cross-lingual equivalents. Adopted multilingual embeddings for retrieval instead, avoiding translation as a mandatory step.

The chatbot is split into two engines behind one interface, coordinated by an intent router — because RAG alone answers policy/explanation questions well but cannot reliably drive a structured process like KYC or loan-application form-filling.

6.2 Engine Split

Engine	Handles	Example
Slot-Filling (Task Dialogue)	Structured processes: KYC, loan application, onboarding	"What's your PAN number?" -> validate -> next field
Multilingual RAG	Open-ended policy/explanation questions	"Why do you need my PAN?" / "What is a cooling-off period?"

6.3 Slot-Filling Engine

- **Deterministic validation, not LLM judgment:** PAN regex, Aadhaar checksum (Verhoeff algorithm, never storing/logging full number), income-range sanity checks, loan-amount eligibility caps. Compliance-critical fields must be validated by code, not model inference.
- **Dialogue state tracker:** Tracks per-journey slot schema (e.g., loan application: name, PAN, Aadhaar last 4, monthly income, loan amount, employment type), current slot, and retry count. State persists even if the user detours into a RAG-answered question mid-flow.
- **Plain-language error correction:** On validation failure, the system explains the correction in the user's language (e.g., "PAN should be 10 characters, like ABCDE1234F") rather than a generic "invalid input" message.
- **Cross-link to Sub-problem 1:** When a loan amount is entered, it passes through the same affordability/stress veto layer used by the recommendation engine before the application can proceed — one veto layer, two entry points.

6.4 Multilingual RAG Engine

- **Embedding model:** multilingual-e5 or LaBSE — embeds vernacular queries directly into a shared vector space with document embeddings, eliminating the translate-before-retrieval failure point.
- **Corpus:** RBI Financial Awareness Messages, PMJDY materials, NPCI/UPI guides, bank product T&Cs. Language coverage per source must be verified rather than assumed; where a regional-language version does not exist, pre-translate once at ingestion time with IndicTrans2 (AI4Bharat) and QC it — safer than live/on-the-fly translation.
- **Chunking:** 200–400 token chunks with ~50-token overlap, split at semantic/clause boundaries rather than fixed character counts, since financial documents carry clause-level meaning.
- **Vector store:** FAISS (local) for hackathon scope; note Pinecone/Weaviate as the production scaling path.
- **Generation instruction:** LLM is prompted to answer in the detected input language, cite the source document, and explicitly decline to guess if retrieved context doesn't contain the answer — critical in a lending/compliance context where a hallucinated answer about loan terms is a regulatory liability, not just a UX bug.

6.5 Intent Router

A lightweight classifier (few-shot LLM prompt, or a fine-tuned multilingual-BERT classifier for lower latency) decides per-turn — not per-session — whether an utterance is task-intent or knowledge-intent, since a user can detour mid-KYC to ask a policy question and then resume the form without losing state.

6.6 Build Priority

- One journey only for the demo (recommend: loan application, given its link to the veto layer).
- Rule-based/keyword intent router for the demo (e.g., "kya/why/matlab" -> RAG intent) rather than a trained classifier, given time constraints.
- RAG over a small subset (5–10 documents) of RBI/PMJDY sources — enough to demonstrate grounding and citation.
- Voice (ASR/TTS) and the IndicTrans2 QC pass are stretch goals, only added if time remains after the text-only vernacular flow is solid.

Scope note: Confirm the target demo language now (Hindi vs. a specific regional language) — embedding-model coverage quality varies significantly across Indian languages, and this choice affects RAG reliability more than any other single decision in this sub-problem.

7. Sub-Problem 3 — Fraud & Financial Stress Detection

7.1 Approach

Fraud and financial stress are treated as two distinct detection problems with different math, different data patterns, and different interventions — not conflated into a single anomaly-detection model.

Dimension	Fraud Detection	Financial Stress Detection
Pattern type	Anomalous, high-deviation, often single-event	Gradual, trend-based, multi-week deterioration
Baseline	Deviation from the user's own transaction history	Deviation from the user's own historical trend line
Model	Isolation Forest / PyOD (unsupervised anomaly detection)	Rolling-window regression / trend-slope classifier (e.g., logistic regression on slope features)
Intervention	Empathetic hold and verification, not automatic decline	Restructuring/counseling offer; suppress new credit-product pushes

7.2 Fraud Detection — Detail

- Model: Isolation Forest or the PyOD library (which wraps multiple algorithms — Isolation Forest, AutoEncoder, LOF — behind one API for fast comparison).
- Dataset: PaySim (Kaggle) — a synthetic mobile-money transaction dataset purpose-built to simulate fraud scenarios, well documented.
- Deep-learning anomaly detection (autoencoders, LSTMs) is deliberately avoided unless the team has prior experience with it — the marginal accuracy gain over Isolation Forest is unlikely to be visible in a demo, while the debugging risk is high.

7.3 Financial Stress Detection — Detail

Stress signals are trend-based, not point-anomaly-based, and must be computed as a separate feature set:

- EMI-to-inflow ratio trend, rising over a 3–6 month rolling window (not a single spike)
- Savings-rate decay — percentage of salary credit retained as balance after 30 days, trending down
- Essential-vs-discretionary spend ratio shift — rising share toward essentials (groceries, utilities) signals belt-tightening
- Balance-floor trend — minimum balance per cycle drifting toward zero across successive cycles
- Missed/delayed EMI pattern — a recurring late-but-eventually-paid pattern (distinct from a single miss, which is closer to a fraud/error signal) indicates cash-flow stress rather than default risk
- Credit utilization creep — rising utilization percentage over time even when payments remain on-time

Using anomaly detection (Isolation Forest) for this signal set is deliberately avoided: stress is a trend, not an outlier, and a judge asking why an anomaly-detection model flagged a slow, gradual decline is a question the architecture should not have to field.

7.4 Intervention Design

Stress-flagged users are directed toward restructuring or counseling nudges (e.g., EMI holiday, financial counseling) and are explicitly suppressed from new credit-product recommendation eligibility via the veto layer (Section 8). This is the concrete link between sub-problems 1 and 3 that should be visible as an explicit edge in the architecture diagram, not just mentioned in the ethics section.

8. Cross-Cutting Decision Layer (Veto Logic)

A pure continuously-tunable soft weight was rejected as the sole mechanism: without a guaranteed floor, a high enough base propensity score could theoretically survive the discount and still reach a severely stressed customer — precisely the predatory-nudging failure mode the problem statement warns against. The system instead uses a hybrid: soft weighting across the healthy-to-mild-stress range, and a hard rule-based veto once a defined severity threshold is crossed.

```
stress_score = f(EMI_trend, savings_decay, spend_shift, balance_floor)

if stress_score > HARD_THRESHOLD:
    credit_products = BLOCKED    # hard rule, non-negotiable
    eligible_actions = [counseling_nudge, EMI_restructure_offer]
else:
    propensity_score *= (1 - soft_weight(stress_score))    # continuous discount
```

This gives a defensible answer to the hardest question a judge can ask: "what is the maximum loan-propensity score a severely stressed user can be shown?" — the answer is a fixed, auditable floor, not "it depends on the weight."

Scope note: Threshold logic (percentile-based within the dataset vs. fixed domain thresholds such as EMI ratio > 50%) is not yet finalized and should be decided during build, with the rationale documented for the ethics writeup.

9. Ethical Safeguards & Regulatory Compliance

9.1 DPDP Act — Purpose Limitation

Addressed structurally via the tiered consent model (Section 5.2): data collected for one purpose is not silently repurposed for another without a corresponding consent basis. Tier 2 (explicit life-stage data) is scoped to the specific product category it was collected for.

9.2 Algorithmic Bias

Deliberately excludes features that function as demographic proxies in Indian contexts — e.g., pincode as a proxy for caste or religion in some regions — even where such features might statistically improve model accuracy. The bias-audit section of the final submission should explicitly list excluded features and the reasoning, not just claim the system is "unbiased."

9.3 Avoiding Predatory Nudging

- Hard-threshold veto floor beneath the soft-weighting layer (Section 8) — not a discount-only mechanism.
- Single recommendation surfaced at a time, countering the multi-offer "pop-up" pattern the problem statement identifies as the current failure mode.
- Investment recommendations use transparent, auditable rules rather than a black-box model, given the regulatory sensitivity of investment mis-selling.

9.4 RBI Data Localization

The submission should explicitly state the assumed data residency (even where hackathon-simulated) and note that production deployment would require infrastructure compliant with RBI's data localization norms for payment-system data.

9.5 Explainability by Construction

SHAP values attach to every propensity-model output; RAG answers cite their source document; the veto layer's hard-threshold behavior is deterministic and auditable rather than emergent from model weights. This directly serves the explainability/compliance-readiness judging criterion rather than treating it as a documentation afterthought.

10. Technology Stack Summary

Layer	Technology	Purpose
Segmentation	scikit-learn (KMeans / HDBSCAN)	Behavioral customer segmentation
Propensity Models	LightGBM	Per-product recommendation scoring; native categorical support, fast on small/medium tabular data
Explainability	SHAP	Per-recommendation plain-language justification
CF Baseline (optional)	Surprise (Python)	Lightweight baseline to demonstrate cold-start reasoning for rejecting pure CF
Fraud Detection	PyOD / Isolation Forest	Unsupervised anomaly detection on transaction data
Stress Detection	Rolling-window regression / logistic regression on trend-slope features	Trend-based financial distress detection
Vernacular Embeddings	multilingual-e5 or LaBSE	Cross-lingual retrieval without mandatory translation
Translation (fallback only)	IndicTrans2 (AI4Bharat)	Offline pre-translation of source documents / QC pass only if needed
RAG Vector Store	FAISS (local)	Document retrieval for the hackathon prototype
Prototype UI	Streamlit	Fast interactive dashboard; renders SHAP plots natively
API Layer	FastAPI	Thin service layer between models and UI/frontend
Production Frontend (stretch)	Next.js + Tailwind	Only after backend/model pipeline is functional and demoable

11. USPs & Differentiators

- **Unified decisioning, not three disconnected demos.** A stress-flagged customer is simultaneously and consistently suppressed from credit offers across both the dashboard and the chatbot, because both surfaces read from the same veto layer.
- **Explainability is structural, not cosmetic.** SHAP justifications and RAG citations are generated as part of the output format itself, not assembled afterward for the pitch.
- **Hard compliance floor beneath ML flexibility.** The system can answer the specific, auditable question of maximum exposure to a distressed customer — most personalization demos cannot.
- **Genuinely separated fraud vs. stress detection.** Recognizes these as different statistical problems (point-anomaly vs. trend) rather than a single "risk score," avoiding a common oversimplification.
- **Translation-avoidant multilingual RAG.** Retrieval happens in shared embedding space, not through a fragile translate-then-search pipeline, directly improving reliability for vernacular-first, non-tech-savvy users.
- **Consent-tiered data model.** Built around DPDP purpose-limitation from the start rather than retrofitted into an ethics slide.

12. Datasets

Dataset	Use	Notes
PaySim (Kaggle)	Fraud detection; transaction-level granularity	Synthetic mobile-money data, purpose-built for fraud simulation
Bank Marketing (UCI/Kaggle)	Propensity-to-buy style modeling	Demographic + campaign response data; weaker on transaction-level behavior
Lending Club Loan Data	EMI/default-risk behavior modeling	US-centric — requires reframing to Indian context in the narrative
RBI / PMJDY / NPCI public documents	Multilingual RAG corpus	Verify actual regional-language coverage per document before build; do not assume

Scope note: No single public dataset covers all three sub-problems. Two or more datasets will need to be stitched together, and this simplification should be stated explicitly in the submission rather than glossed over.

13. Build Roadmap & Prioritization

Recommended build order, front-loading shared infrastructure and the two highest-weight sub-problems (personalization, fraud/stress) before the chatbot, and deferring frontend polish until models return real data.

Step	Task	Depends On
1	Shared feature engineering pipeline + stress/fraud veto layer	Dataset selection finalized
2	Loan propensity model + SHAP explanation	Step 1
3	Insurance propensity model	Step 1
4	Fraud detection (Isolation Forest / PyOD) on PaySim	Step 1
5	Stress-trend detection model	Step 1
6	Streamlit dashboard wired to steps 2–5, live veto behavior demoable	Steps 2–5
7	Slot-filling engine for one journey (loan application), linked to veto layer	Step 1
8	Multilingual RAG over 5–10 RBI/PMJDY documents + rule-based intent router	Independent; can parallelize with 7
9 (stretch)	Investment rules engine, credit card model	Steps 2–5 complete with time remaining
10 (stretch)	Next.js frontend port, voice ASR/TTS, IndicTrans2 QC pass	Steps 6–8 complete with time remaining

14. Judging Criteria Mapping

Criterion	How This System Addresses It
Innovation & technical feasibility	Multi-specialist-model architecture on a shared pipeline; feasible within hackathon constraints using lightweight, well-documented libraries (LightGBM, PyOD, FAISS)
Depth of personalization vs. genuine benefit	Hard veto floor prevents credit-push to stressed users; single-recommendation-at-a-time; investment uses transparent rules to avoid mis-selling risk
Explainability & RBI/regulatory readiness	SHAP on every recommendation; RAG citations; deterministic hard-threshold logic; consent-tiered data model aligned to DPDP purpose limitation
Usability for non-tech-savvy, vernacular-first users	Multilingual embedding retrieval (no fragile translation step); plain-language error correction in slot-filling; native-language generation
Scalability across existing infrastructure	FastAPI service layer designed to sit alongside existing banking APIs; FAISS/Streamlit stack has a clear production path (Pinecone/Weaviate, Next.js) without architectural rework

15. Known Limitations & Honest Gaps

- No single public dataset covers all three sub-problems; results rely on stitched datasets that should be disclosed, not hidden.
- Stress-severity thresholds are not yet finalized (percentile-based vs. fixed domain thresholds) — a build-time decision with direct compliance implications.
- Regional-language document coverage for the RAG corpus is not yet verified and may require offline pre-translation for some languages.
- Voice interaction (ASR/TTS) is out of scope for the initial build and is a stretch goal only.
- The chatbot demo covers a single journey (loan application) rather than full KYC/onboarding coverage; the architecture is designed to generalize via additional slot schemas but this is not yet built.
- Credit card and investment product models are lower priority and may remain at the design/architecture-diagram stage rather than fully implemented, depending on remaining time.